Ultra runs: ultracode and ultrareview, spent well

Claude Code has two "go big" modes, and they're easy to mix up.

Kind Pattern | Changed 2026-11-04 | For Anyone with three free runs to spend

You are here: drasticstatic lab › `my-template` › `walkthroughs` › `ULTRA-RUNS.md` · public · `main` is the working branch

TL;DR

1. **Ultracode** makes your helper run *every* substantive task as a team of agents. It's a switch, it uses your plan's usage, and it has no free runs.
2. **Ultrareview** sends a team of reviewers into the cloud to find and double-check bugs. It never edits code. Pro and Max accounts get **three free runs, once, ever**.
3. **The trap:** ultrareview reviews the *difference* between your branch and `main`. If all the work is already on `main`, that difference is empty and the free run is gone. Stage a review first (below).

> ***Handing this to your AI helper?*** *Say: "Read ULTRA-RUNS.md. Help me stage a review for and tell me when it's ready. Don't start the review yourself." (Helpers can't start an ultrareview; you do.)*

Ultracode

Turn on	`/effort ultracode` (or in the `/effort` slider, press `Tab` to flip **Ultracode**, then `Enter`)
Turn off	`/effort ultracode off`
Effort	Works at whatever effort is set. Set it first (e.g. `/effort high`). `claude --effort ultracode` also sets `xhigh`.
Cost	Draws on your plan's usage limits; you'll reach session and weekly limits sooner.
Good for	Deep reconnaissance across many repos, audits, migrations, big polish passes.

Before you turn it on:

1. Everything committed and pushed; every repo you'll touch is synced.
2. The repo has a `CLAUDE.md` with its rules. Every agent the run starts reads it, so this is where the rules that must never break go.
3. A written brief: what to look at, what "done" means, what's out of scope.
4. Connected tools (MCP servers, the browser extension) signed in **before** the run, not during it.
5. Approve edits by hand until you're ready, so the switch doesn't flip early.

Who publishes: the main session, not the agents

Publishing and repository creation run in the main session; subagents prepare. In a multi-agent run, the safety classifier refuses `gh repo create` (and can refuse a first push to a new public repository) when a background agent attempts it, even where `~/.claude/settings.json` allows the command. Plan for it:

Agents do	The main session does, with the person approving each prompt
write, commit and push to repositories that already exist; open pull requests	`gh repo create`, a repository's first public push, enabling Pages
stage a new repository in a scratch folder, with its commits and a `CREATE-AND-PUSH.sh`	run that script, then move the clone to its home
list the exact commands they could not run (`main_session_steps`)	run them in small batches and read each result back
prepare deploy-key and ruleset commands	generate keys, add deploy keys, set secrets, apply rulesets

A run that ignores this ends with work that is staged but not live, and a list of refusals to replay by hand. Write the brief so each agent returns its unfinished steps as exact commands.

Ultrareview

Start	`/code-review ultra` (your branch vs `main`) or `/code-review ultra <PR number>` (a GitHub pull request)
Time	About 15–25 minutes, in the cloud
Free runs	Three, one-time per account. **A run you stop early, or that fails, still uses one.**
Output	Verified findings in your session. It doesn't change code. Add `--post` on a PR to post them there.

Stage a review so the run sees the code

When the work is already on `main`, give the review a base that doesn't have the code yet:

```
git switch -c review/code-baseline main
git rm -r -q <the code folders to review>              # e.g. src/ scripts/ tests/ .github/ (main brings them back)
git commit -m "review baseline: main without the code under review (never merge)"
git push -u origin review/code-baseline
git switch main
gh pr create --draft --base review/code-baseline --head main \
  --title "Ultrareview target: <what> (do not merge)" --body "Review-only PR. Diff = the code under review."
gh pr diff <number> --name-only | wc -l                  # not zero, and only the files you meant
```

Then `/code-review ultra <number>`. The PR follows `main`, so it stays a valid review target as work lands. Keep lockfiles, images and generated files in the baseline so the reviewers spend their attention on code.

Choosing what to spend the three on

Spend them on code that is **large, live, and hard to test by hand**: a site people use, contract or payment code, CI that publishes. A good order is: recon first (what is where, and what's risky), then stage a PR per target, then run the reviews one at a time and fix each one's findings before the next.

Suggest a change

Noticed something? Open an issue or a pull request against `main`, changing this page; a steward reads it and a person with the role merges, with a merge commit, so your name stays on it. An AI helper may do all of that up to the merge, signed as the attribution spec says. Not on GitHub? Tell any steward you know, or your helper, in your own words.

Plan limits, free-run counts and what a run costs are Anthropic's to set and change; check their own pages before relying on a number here. This is a description of how the modes behaved when written, not advice on what to spend. Part of the walkthroughs. Icons: Phosphor duotone (MIT).